# HW2

Group 12 - Shir, Omer & Ido

14/04/26

**Group:**

Group members (Name, ID, email)


- Ido Ravid, 207198177, ido.ravid@mail.huji.ac.il
- Omer Sutovsky, 318735107 ,omer.sutovsky@mail.huji.ac.il
- Shir Tsarfaty, 207718008, shir.tsarfaty@mail.huji.ac.il


**Paths and Data**

```r
if (!require('data.table')){
  install.packages('data.table')
  library('data.table')
}
```

```
## Loading required package: data.table
```

```r
library('tictoc')
```

```
##
## Attaching package: 'tictoc'

## The following object is masked from 'package:data.table':
##
##     shift
```

```r
if (!require('manipulate')){
  install.packages('manipulate')
  library('manipulate')
}
```

```
## Loading required package: manipulate
```

```r
reads_file = "data/TCGA-13-0723-10B_lib2_all_chr1.forward"

beg_region <- 50000000
end_region <- 60000000
```

**Reading the Data:**

```r
chr1_reads = fread(reads_file, col.names = c("Chr", "Loc", "Length"))
```

## Introduction

This lab focuses on analyzing NGS fastq files, and specifically read coverage and depth.

## Part 1

### Question 1

The original, semi-naive implelmtatnion here is adapted from Yuval's second lecture, with additional logging for easier comparison.

```r
count_reads_naive <- function(reads, beg_region, end_region, log_every = 1e6) {
  N <- end_region - beg_region + 1
  read_line <- numeric(N)

  first_read <- min(which(reads$Loc >= beg_region))
  r <- first_read
  start_time <- proc.time()["elapsed"]

  while (reads$Loc[r] <= end_region) {
    read_line[reads$Loc[r] - beg_region + 1] <- read_line[reads$Loc[r] - beg_region + 1]
↪  + 1
    if ((r - first_read) %% log_every == 0) {
      elapsed <- round(proc.time()["elapsed"] - start_time, 1)
      cat("Progress: read", r, "| loc", reads$Loc[r], "| elapsed:", elapsed, "s\n")
      flush.console()
    }
    r <- r + 1
  }
  read_line
}
```

```r
tic()
counts_naive <- count_reads_naive(chr1_reads, beg_region, end_region)
```

```
## Progress: read 2029949 | loc 50000051 | elapsed: 0 s
```

```r
t_naive <- toc(quiet = TRUE)

message("Total fragments in region: ", sum(counts_naive))
```

```
## Total fragments in region: 354584
```

```r
message("Time taken: ", round(t_naive$toc - t_naive$tic, 2), "s")
```

```
## Time taken: 0.51s
```

### Question 2

To find a better implementaiton, we looked at several options suggested in StackOverflow (links below), before finally settling on this one that uses `tabulate`.

We rely mostly on the assumption that the location list is already sorted (if it wasn't, it would have been an O(nlogn) task instead). The reason for our expectation for this to be faster is empirical- we tried several options mentioned in StackOverflow, and this one seemed to do the trick.

```r
count_reads_table <- function(reads, beg_region, end_region) {
  t0 <- proc.time()["elapsed"]

  # keep only reads that fall within the region
  locs <- reads$Loc[reads$Loc >= beg_region & reads$Loc <= end_region]
  message("Filtering done: ", length(locs), " reads in region | ",
          round(proc.time()["elapsed"] - t0, 2), "s")

  # tabulate uses 1-based indexing, so shift locs to be relative to beg_region
  counts <- tabulate(locs - beg_region + 1, nbins = end_region - beg_region + 1)
  result <- counts
  message("Done | total: ", round(proc.time()["elapsed"] - t0, 2), "s")

  result
}
```

```r
tic()
counts_table <- count_reads_table(chr1_reads, beg_region, end_region)
```

```
## Filtering done: 354584 reads in region | 0.03s
```

```
## Done | total: 0.03s
```

```r
t_table <- toc(quiet = TRUE)

message("Total fragments in region: ", sum(counts_table))
```

```
## Total fragments in region: 354584
```

```r
message("Time taken: ", round(t_table$toc - t_table$tic, 2), "s")
```

```
## Time taken: 0.04s
```

### Question 3

To compare the functions, we create a benchmark test. the tests randomly selects a 1M window for 100 times. it run each function over that window, and stores the result. Here is the benchmark:

```r
chr1_max_loc <- 247199205
window_size  <- 1e6
n_runs       <- 100

times_naive  <- numeric(n_runs)
times_tabulate <- numeric(n_runs)

set.seed(42)
starts <- sample(1:(chr1_max_loc - window_size), n_runs)

for (i in 1:n_runs) {
  beg <- starts[i]
  end <- beg + window_size - 1

  t0 <- proc.time()["elapsed"]
  suppressMessages(count_reads_naive(chr1_reads, beg, end))
  times_naive[i] <- proc.time()["elapsed"] - t0

  t0 <- proc.time()["elapsed"]
  suppressMessages(count_reads_table(chr1_reads, beg, end))
  times_tabulate[i] <- proc.time()["elapsed"] - t0

  if (i %% 10 == 0) { cat("Run", i, "/", n_runs, "\n"); flush.console() }
}
```

```
## Progress: read 5005555 | loc 170979319 | elapsed: 0 s
## Progress: read 4511577 | loc 155243699 | elapsed: 0 s
## Progress: read 2725020 | loc 71927241 | elapsed: 0 s
## Progress: read 6178338 | loc 210838275 | elapsed: 0 s
## Progress: read 3076397 | loc 86947907 | elapsed: 0 s
## Progress: read 3567028 | loc 106492092 | elapsed: 0 s
## Progress: read 5124154 | loc 174923327 | elapsed: 0 s
## Progress: read 4708478 | loc 161058730 | elapsed: 0 s
## Run 10 / 100
## Progress: read 6465238 | loc 220787311 | elapsed: 0 s
## Progress: read 3043071 | loc 85623722 | elapsed: 0 s
## Progress: read 2551023 | loc 65333217 | elapsed: 0 s
## Progress: read 1678627 | loc 40687115 | elapsed: 0 s
## Progress: read 6304858 | loc 215207848 | elapsed: 0 s
## Progress: read 2257637 | loc 56340545 | elapsed: 0 s
## Progress: read 670360 | loc 16963307 | elapsed: 0 s
## Progress: read 1296943 | loc 31470892 | elapsed: 0 s
## Progress: read 794372 | loc 19558335 | elapsed: 0 s
```

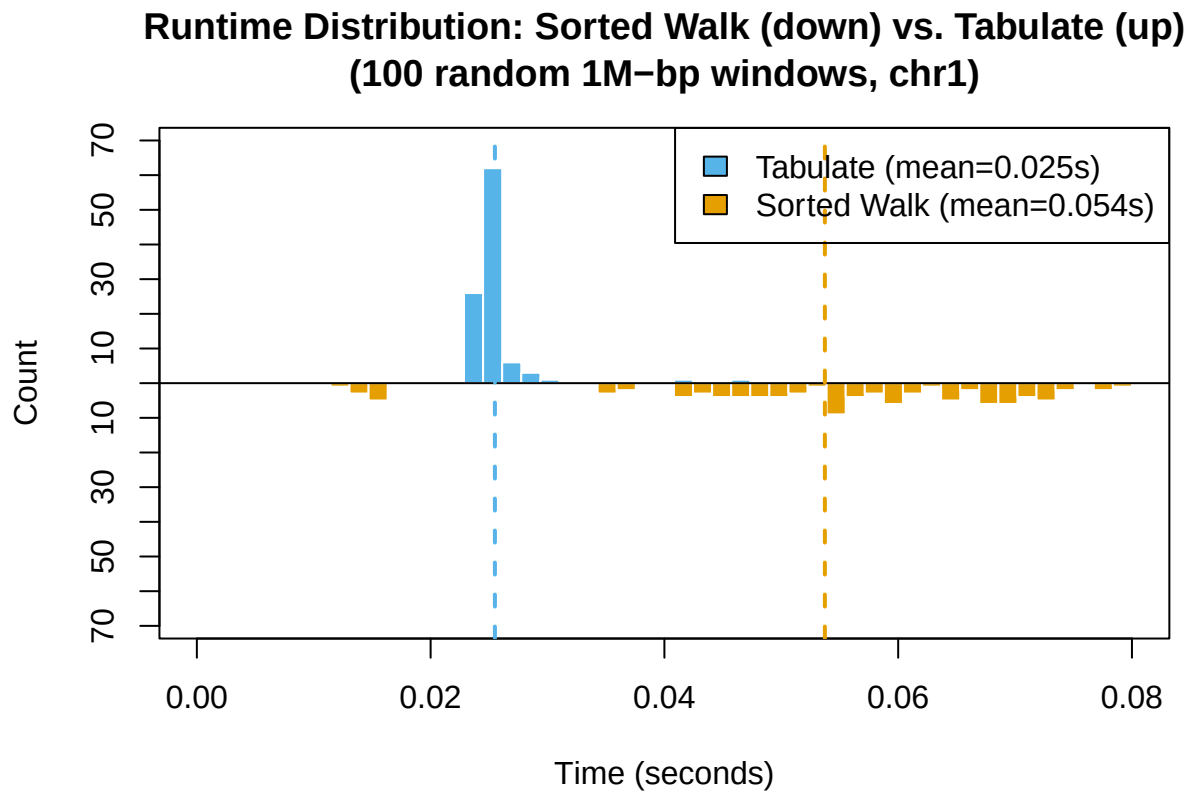
```
## Run 20 / 100
## Progress: read 4701197 | loc 160823716 | elapsed: 0 s
## Progress: read 7241718 | loc 243791159 | elapsed: 0 s
## Progress: read 2198625 | loc 54764535 | elapsed: 0 s
## Progress: read 4134360 | loc 143761015 | elapsed: 0 s
## Progress: read 3158510 | loc 89872602 | elapsed: 0 s
## Progress: read 3464732 | loc 100911378 | elapsed: 0 s
## Progress: read 4903819 | loc 167231419 | elapsed: 0 s
## Progress: read 6591371 | loc 224537618 | elapsed: 0 s
## Progress: read 1947667 | loc 47612960 | elapsed: 0 s
## Progress: read 6362271 | loc 217381801 | elapsed: 0 s
## Run 30 / 100
## Progress: read 1690164 | loc 40997127 | elapsed: 0 s
## Progress: read 4665722 | loc 159723651 | elapsed: 0 s
## Progress: read 3206029 | loc 91477035 | elapsed: 0 s
## Progress: read 1001174 | loc 24460703 | elapsed: 0 s
## Progress: read 5434668 | loc 184624458 | elapsed: 0 s
## Progress: read 1753872 | loc 42580131 | elapsed: 0 s
## Progress: read 1398371 | loc 33841191 | elapsed: 0 s
## Progress: read 2439292 | loc 61866142 | elapsed: 0 s
## Progress: read 2952844 | loc 81995899 | elapsed: 0 s
## Run 40 / 100
## Progress: read 5762408 | loc 199861584 | elapsed: 0 s
## Progress: read 3697496 | loc 110923862 | elapsed: 0 s
## Progress: read 5703523 | loc 198383386 | elapsed: 0 s
## Progress: read 3421051 | loc 99241514 | elapsed: 0 s
## Progress: read 5368191 | loc 182190133 | elapsed: 0 s
## Progress: read 3177527 | loc 90500833 | elapsed: 0 s
## Progress: read 6366638 | loc 217562129 | elapsed: 0 s
## Progress: read 3118286 | loc 88376284 | elapsed: 0 s
## Progress: read 7197698 | loc 242589331 | elapsed: 0 s
## Progress: read 188907 | loc 5322720 | elapsed: 0 s
## Run 50 / 100
## Progress: read 867315 | loc 21296794 | elapsed: 0 s
## Progress: read 6607373 | loc 224902085 | elapsed: 0 s
## Progress: read 4438568 | loc 153551003 | elapsed: 0 s
## Progress: read 5744260 | loc 199480944 | elapsed: 0 s
## Progress: read 242476 | loc 6517842 | elapsed: 0 s
## Progress: read 4444465 | loc 153717276 | elapsed: 0 s
## Progress: read 214938 | loc 5921573 | elapsed: 0 s
## Progress: read 6256684 | loc 213177398 | elapsed: 0 s
## Progress: read 5532076 | loc 189950323 | elapsed: 0 s
## Run 60 / 100
## Progress: read 1932418 | loc 47159107 | elapsed: 0 s
## Progress: read 5530326 | loc 189855502 | elapsed: 0 s
## Progress: read 5687251 | loc 197704385 | elapsed: 0 s
## Progress: read 1151656 | loc 27979771 | elapsed: 0 s
## Progress: read 4767210 | loc 163157144 | elapsed: 0 s
## Progress: read 1580116 | loc 38214960 | elapsed: 0 s
## Progress: read 2452710 | loc 62300075 | elapsed: 0 s
## Progress: read 3457871 | loc 100668665 | elapsed: 0 s
## Run 70 / 100
## Progress: read 3805687 | loc 114149259 | elapsed: 0 s
## Progress: read 5687343 | loc 197710217 | elapsed: 0 s
```

```
## Progress: read 5612424 | loc 193983242 | elapsed: 0 s
## Progress: read 5487970 | loc 187311701 | elapsed: 0 s
## Progress: read 6807671 | loc 230213288 | elapsed: 0 s
## Progress: read 29418 | loc 1574793 | elapsed: 0 s
## Progress: read 1889806 | loc 46050895 | elapsed: 0 s
## Progress: read 4940230 | loc 168637050 | elapsed: 0 s
## Run 80 / 100
## Progress: read 4343500 | loc 151334534 | elapsed: 0 s
## Progress: read 2435618 | loc 61755779 | elapsed: 0 s
## Progress: read 6817941 | loc 230502335 | elapsed: 0 s
## Progress: read 5339319 | loc 181365123 | elapsed: 0 s
## Progress: read 4916990 | loc 167738125 | elapsed: 0 s
## Progress: read 6627133 | loc 225452846 | elapsed: 0 s
## Progress: read 2101238 | loc 52336590 | elapsed: 0 s
## Progress: read 5546164 | loc 190694637 | elapsed: 0 s
## Progress: read 5442104 | loc 184933691 | elapsed: 0 s
## Progress: read 779341 | loc 19219133 | elapsed: 0 s
## Run 90 / 100
## Progress: read 6987357 | loc 235257699 | elapsed: 0 s
## Progress: read 5426432 | loc 184273806 | elapsed: 0 s
## Progress: read 1473061 | loc 35735982 | elapsed: 0 s
## Progress: read 1269619 | loc 30853316 | elapsed: 0 s
## Progress: read 646553 | loc 16589076 | elapsed: 0 s
## Progress: read 1727566 | loc 41881003 | elapsed: 0 s
## Progress: read 7094700 | loc 239188839 | elapsed: 0 s
## Progress: read 1785340 | loc 43484329 | elapsed: 0 s
## Progress: read 4959318 | loc 169374118 | elapsed: 0 s
## Run 100 / 100
```

And the results:

```r
x_max <- 0.08
breaks <- seq(0, x_max, length.out = 50)
h_naive    <- hist(pmin(times_naive, x_max),    breaks = breaks, plot = FALSE)
h_tabulate <- hist(pmin(times_tabulate, x_max), breaks = breaks, plot = FALSE)

y_max <- max(h_naive$counts, h_tabulate$counts) * 1.1
plot(0, type = "n", xlim = c(0, 0.08), ylim = c(-y_max, y_max),
     xlab = "Time (seconds)", ylab = "Count",
     main = "Runtime Distribution: Sorted Walk (down) vs. Tabulate (up)\n(100 random
     ↪  1M-bp windows, chr1)",
     yaxt = "n")

yticks <- pretty(c(0, y_max))
yticks <- yticks[yticks > 0]  # exclude 0 to avoid duplication at center
axis(2, at = c(-yticks, 0, yticks), labels = c(yticks, 0, yticks))

rect(h_tabulate$breaks[-length(h_tabulate$breaks)], 0,
     h_tabulate$breaks[-1], h_tabulate$counts,
     col = "#56B4E9", border = "white")
rect(h_naive$breaks[-length(h_naive$breaks)], 0,
     h_naive$breaks[-1], -h_naive$counts,
     col = "#E69F00", border = "white")
```

```r
abline(h = 0, lwd = 1)
abline(v = mean(times_naive),    col = "#E69F00", lwd = 2, lty = 2)
abline(v = mean(times_tabulate), col = "#56B4E9", lwd = 2, lty = 2)

legend("topright",
       legend = c(sprintf("Tabulate (mean=%.3fs)", mean(times_tabulate)),
                  sprintf("Sorted Walk (mean=%.3fs)", mean(times_naive))),
       fill = c("#56B4E9", "#E69F00"))
```



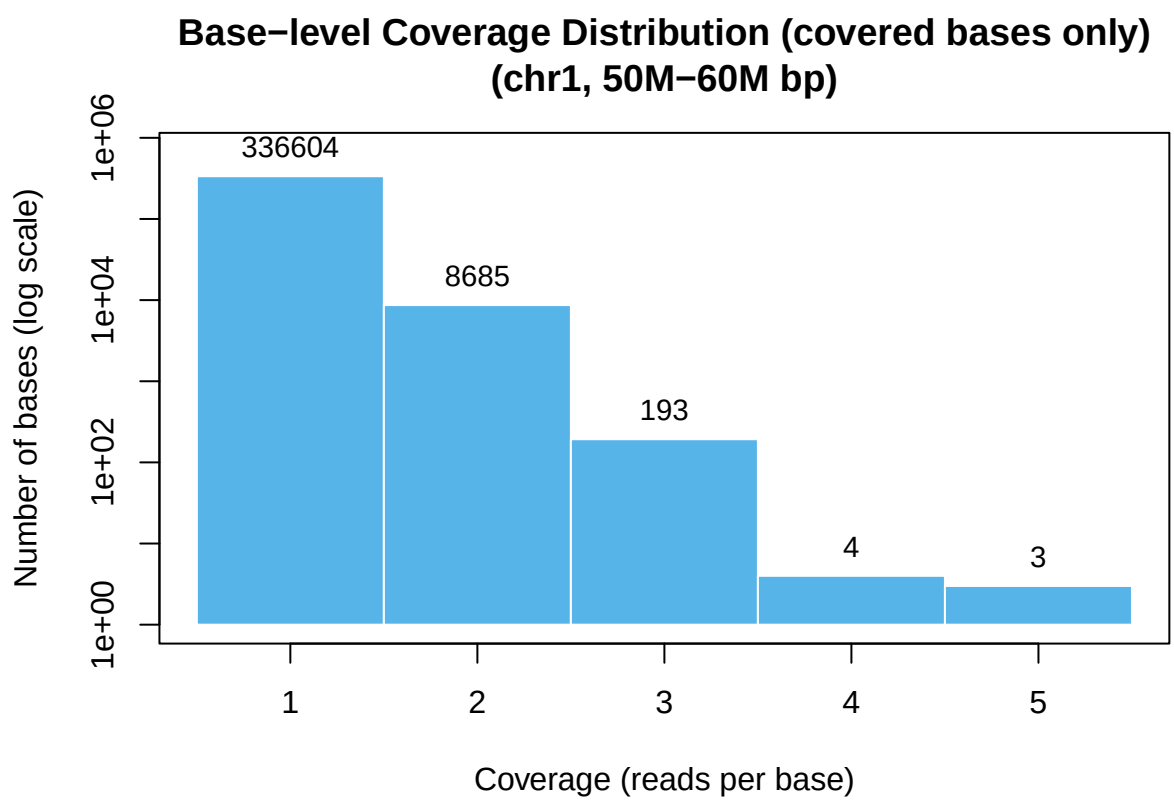

We can clearly see that the tabulate algorithm is superior to the naive sorted-walk algorithm seen in the lecture.

**Question 4**

As we've done in the lecture, creating a histogram of the marginal coverage distribution:

```r
covered <- counts_table[counts_table > 0]
max_cov <- min(max(covered), 50)

read_hist <- hist(pmin(covered, max_cov),
                  breaks = seq(0.5, max_cov + 0.5, by = 1),
                  plot = FALSE)

# log="y" in plot.histogram drops bars - draw manually
plot(read_hist$mids, read_hist$counts,
```

```r
     type = "n", log = "y",
     xlim = range(read_hist$breaks),
     ylim = c(1, max(read_hist$counts) * 2),
     xlab = "Coverage (reads per base)",
     ylab = "Number of bases (log scale)",
     main = sprintf("Base-level Coverage Distribution (covered bases only)\n(chr1,
     ↪  %dM-%dM bp)",
                    beg_region/1e6, end_region/1e6),
     xaxt = "n")

rect(read_hist$breaks[-length(read_hist$breaks)], 1,
     read_hist$breaks[-1], read_hist$counts,
     col = "#56B4E9", border = "white")

axis(1, at = 1:max_cov)

text(read_hist$mids, read_hist$counts, read_hist$counts, pos = 3, cex = 0.9)
```




Note that we log-scaled the graph so the 1st bin will fit. We can see the same tailed distribution we saw in class.

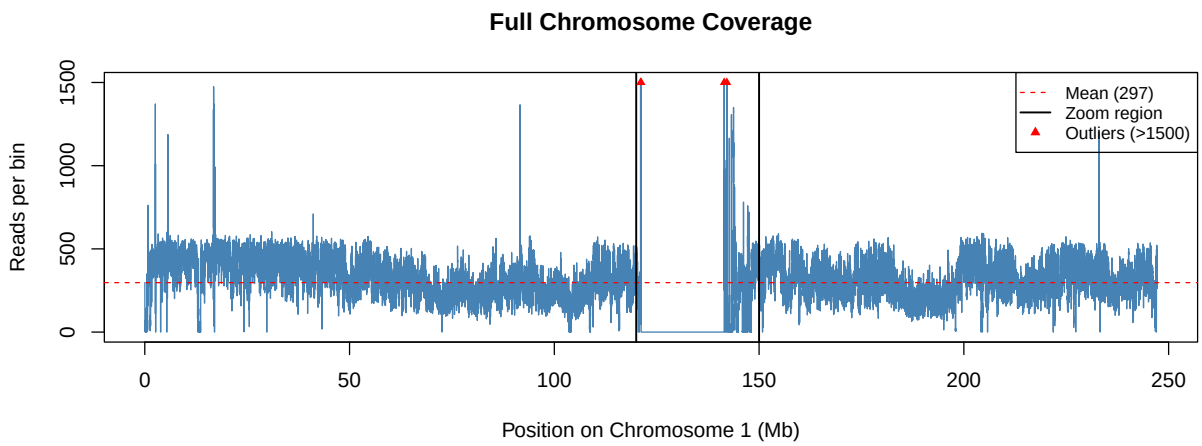
## Part 2

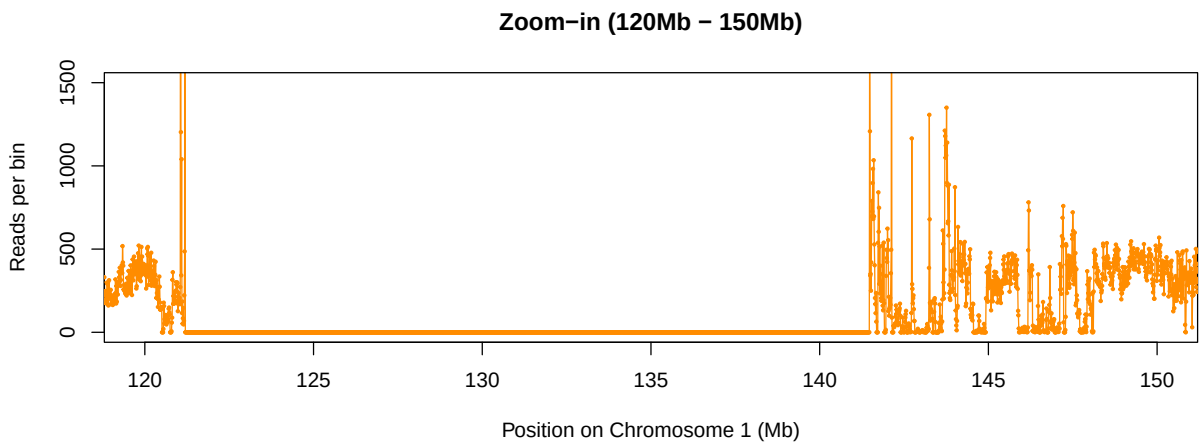
**Question 5.1 & 5.2:**

```r
# Define the bin size (must be a multiple of 10,000)
BIN_SIZE <- 10000

# Function to summarize base-level coverage into genomic bins
bin_coverage <- function(base_counts, bin_size) {
  n_bins <- floor(length(base_counts) / bin_size)
  counts_trimmed <- base_counts[1:(n_bins * bin_size)]

  # Reshape into matrix and calculate column sums for efficiency
  bin_matrix <- matrix(counts_trimmed, nrow = bin_size)
  bin_sums <- colSums(bin_matrix)

  return(bin_sums)
}
```

```r
# Compute full-chromosome coverage using tabulate (no region filter needed)
full_chr_counts <- tabulate(chr1_reads$Loc)
binned_data <- bin_coverage(full_chr_counts, BIN_SIZE)
bin_positions_mb <- (seq_along(binned_data) * BIN_SIZE) / 1e6

zoom_start <- 120
zoom_end <- 150
y_cap <- 1500

par(mfrow = c(2, 1))

# 5.1: Full chromosome coverage plot (capped y-axis, outliers marked)
plot(bin_positions_mb, pmin(binned_data, y_cap),
     type = "l", col = "steelblue",
     xlab = "Position on Chromosome 1 (Mb)",
     ylab = "Reads per bin",
     main = "Full Chromosome Coverage",
     ylim = c(0, y_cap))
abline(h = mean(binned_data), col = "red", lty = 2)
abline(v = c(zoom_start, zoom_end), col = "black", lty = 1, lwd = 1.5)
# mark outlier bins above cap
outliers <- which(binned_data > y_cap)
points(bin_positions_mb[outliers], rep(y_cap, length(outliers)),
       col = "red", pch = 17, cex = 0.7)
legend("topright",
       legend = c(sprintf("Mean (%.0f)", mean(binned_data)), "Zoom region",
                  sprintf("Outliers (>%d)", y_cap)),
       col = c("red", "black", "red"), lty = c(2, 1, NA),
       pch = c(NA, NA, 17), lwd = c(1, 1.5, NA), cex = 0.8)

# 5.2: Zoom-in plot (20Mb window)
plot(bin_positions_mb, binned_data,
     type = "l", col = "darkorange",
     xlim = c(zoom_start, zoom_end),
```

```r
     ylim = c(0, y_cap),
     xlab = "Position on Chromosome 1 (Mb)",
     ylab = "Reads per bin",
     main = sprintf("Zoom-in (%dMb - %dMb)", zoom_start, zoom_end))
points(bin_positions_mb, binned_data, pch = 20, cex = 0.5, col = "darkorange")
```



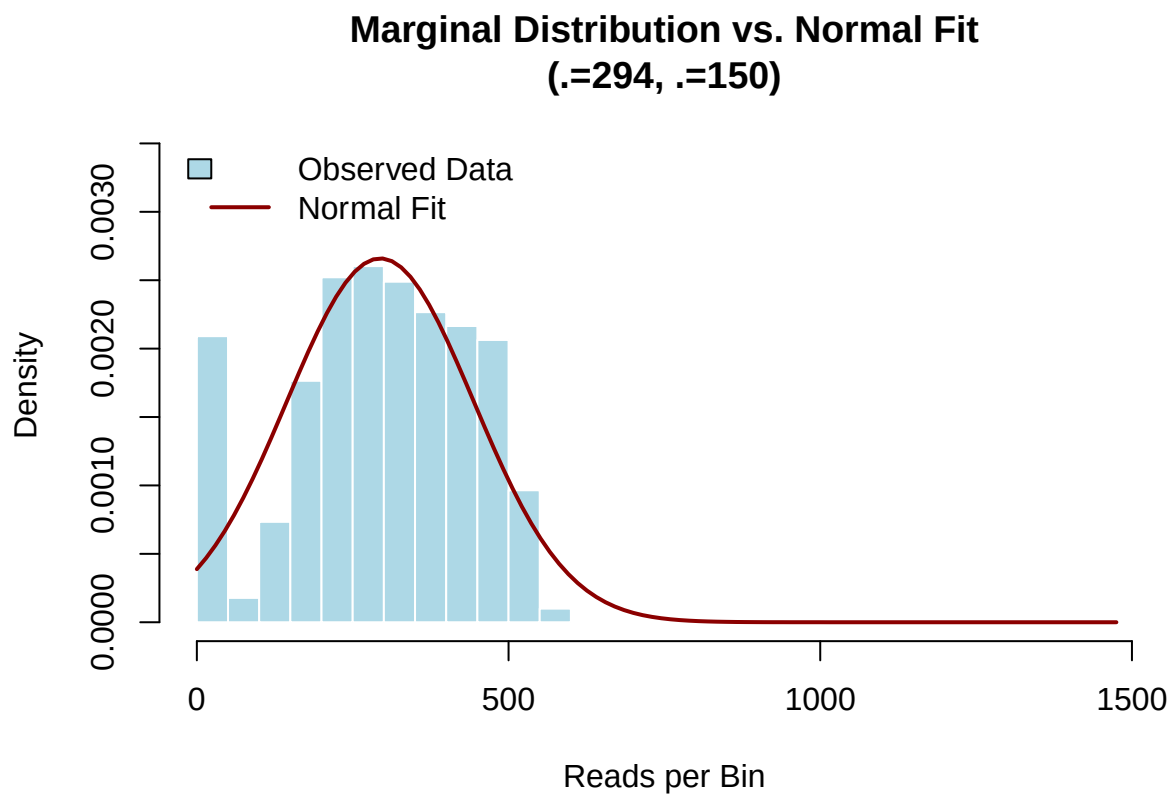

## Discussion

Based on the plots, we can spot several anomalies. In the first plot of the full chromosome, the area around 120-130Mb shows a clear gap with coverage of almost 0, which is related to the centeromer we mentioned in class.

Right after the this part (~130-145Mb), we see extreme outlier spikes, likely caused by reads mapping to duplicated segments. Additionally, the beginning of the chromosome (0-30Mb) also shows high coverage above the mean, which might indicate that several segments there were also copied multiple times.

### Question 6

```r
# Filter out extreme outlier bins before fitting
binned_no_outliers <- binned_data[binned_data <= y_cap]

# Calculate parameters for the Normal distribution
mu <- mean(binned_no_outliers)
sigma <- sd(binned_no_outliers)

# Pre-calculate histogram and density curve to set proper axis limits
h_data <- hist(binned_no_outliers, breaks = 30, plot = FALSE)
x_vals <- seq(min(binned_no_outliers), max(binned_no_outliers), length = 100)
y_vals <- dnorm(x_vals, mean = mu, sd = sigma)

# Plot the histogram using freq = FALSE to correctly show density
# This fixes the warning and ensures bars align with the normal curve
plot(h_data, freq = FALSE,
     col = "lightblue", border = "white",
     xlab = "Reads per Bin",
     ylab = "Density",
     main = sprintf("Marginal Distribution vs. Normal Fit\n(\u03bc=%.0f, \u03c3=%.0f)",
     ↪  mu, sigma),
     ylim = c(0, max(h_data$density, y_vals) * 1.3))

# Overlay the Normal density curve
lines(x_vals, y_vals, col = "darkred", lwd = 2)

# Add legend to the top-left to avoid overlapping with the distribution peak
legend("topleft", legend = c("Observed Data", "Normal Fit"),
       fill = c("lightblue", NA), border = c("black", NA),
       lty = c(NA, 1), col = c(NA, "darkred"), lwd = c(NA, 2),
       bty = "n")
```




The normal distribution does not seem to fit well at all - we could argue that the middle is somewhat normal, but even that doesn't seem correct, as it's not really bell-shaped. the main differences are in the heavy left tail of the distribution, and in the missing right tail of it. also the middle part looks more uniform then we'd expect.

### Short summary

In the first part, we experimented with optimizing the vector-reading function we saw in class. using the built-in , C-based functions we managed to do that pretty well.

In the second part, we mapped the coverage across the entire chromosome using the NGS data. we looked for interesting and irregular regions, and tried to compare the distribution to a normal one.

### External sources statement

We read a few articles in StackOverflow to find reccomendations for optimizing the R function. Specifically relyed on this article, and this specific comment.

We also used AI (Claude) to:

1. help us correctly use the tabulate syntax in question 2
2. To create the back-to-back histogram using rectangles
3. help us debug missing rectangle bug in question 4 after using log transformation.
4. help us implement an efficient binning strategy using matrix operations and colSums.
5. Assist with the multi-panel plot layout and fine-tuning the legend positioning to ensure all data remained visible.